Unsupervised learning exam [MCQ] (Version : 0)

TEST

Correct Answer

Answered in 7.78 Minutes

Question 1/39

Which of the following steps is NOT part of the K-means clustering algorithm?

- [ ] Assign each observation to the nearest centroid.
- [ ] Update the cluster centroids based on the assigned observations.
- [x] Compute the silhouette score for each observation.
- [ ] Randomly initialise K cluster centroids.

Explanation:

Compute the silhouette score for each observation: This option is correct because computing the silhouette score is not a step in the K-means clustering algorithm itself. The silhouette score is a method for evaluating the quality of the clusters formed by the algorithm, not part of the algorithm's iterative process.

Randomly initialise K cluster centroids.: This option is incorrect because it is the first step of K-means.

Assign each observation to the nearest centroid.: This option is incorrect because it is part of the iterative process of K-means.

Update the cluster centroids based on the assigned observations: This option is incorrect because

updating centroids is part of the iterative process in K-means.

Question 2/39

What does the following line of code achieve?

centers = scaler.inverse_transform(km.cluster_centers_)

where

km = KMeans(n_clusters=K, random_state=42)

scaler = StandardScaler()

- [x] It transforms the cluster centres back to the original feature space
- [ ] It scales the cluster centres to have a mean of 0 and a standard deviation of 1
- [ ] It reduces the dimensionality of the cluster centres
- [ ] It predicts the cluster centres for the given data

Explanation:

It transforms the cluster centres back to the original feature space**:** This option is correct because inverse_transform reverses the scaling applied by the scaler, thus transforming the cluster centres from the scaled feature space back to the original feature space.

It scales the cluster centres to have a mean of 0 and a standard deviation of 1.

This option is incorrect because Scaling to mean 0 and std 1 is done using fit_transform, not inverse_transform.

It reduces the dimensionality of the cluster centres.

This option is incorrect because dimensionality reduction is not the goal but rather transforming the cluster centres back to the original feature space.

It predicts the cluster centres for the given data.

This option is incorrect because the prediction of cluster centres is not applicable; transformation is the key action.

Question 3/39

Which of the following metrics would you use to evaluate the compactness of clusters in K-means?

- [ ] R-squared
- [ ] Precision and Recall
- [x] Silhouette Score
- [ ] Mean Squared Error

Explanation:

Silhouette Score: This option is correct because the silhouette score measures how similar each point is to its own cluster compared to other clusters, which directly evaluates the compactness and separation of clusters.

Mean Squared Error: Mean Squared Error is typically used in regression, not clustering.

R-squared: R-squared is used in regression to measure the goodness of fit.

Precision and Recall: Precision and Recall are metrics used for classification tasks.

Question 4/39

In the context of the Elbow Method and K-means clustering, what does the 'elbow' in the plot represent?

- [ ] The point where the number of clusters is equal to the number of data points
- [x] The point where adding more clusters does not significantly reduce the within-cluster sum of squares (WCSS)
- [ ] The point where the within-cluster sum of squares (WCSS) is minimised
- [ ] The point where the within-cluster sum of squares (WCSS) is maximised

Explanation:

The point where adding more clusters does not significantly reduce the WCSS: This option is correct because the 'elbow' represents the point in the plot where adding additional clusters results in a diminishing reduction of the Within-Cluster Sum of Squares (WCSS), indicating the optimal number of clusters.

The point where the WCSS is maximised: the WCSS is not maximised in effective clustering. Instead, we aim to reduce WCSS, but this process continues without significant benefit beyond a certain point, known as the 'elbow'.

The point where the number of clusters is equal to the number of data points:
This option is incorrect because the number of clusters being equal to data points is impractical and not useful.

The point where the WCSS is minimised: This option is incorrect because WCSS continues to decrease with more clusters, but the optimal point is where it decreases less significantly.

Question 5/39

What does the inertia_ attribute of a fitted K-means model represent?

- [ ] The number of clusters used in the model
- [ ] The number of iterations the algorithm ran before converging
- [x] The sum of squared distances of samples to their closest cluster centre
- [ ] The average distance between cluster centres

Explanation:

The sum of squared distances of samples to their closest cluster centre: This option is correct because inertia_ is defined as the sum of squared distances from each point to its assigned cluster centre, measuring how tightly the clusters are packed.

The number of iterations the algorithm ran before converging: This option is incorrect because the number of iterations is not related to inertia.

The average distance between cluster centres: This option is incorrect because the average distance between cluster centres is not the definition of inertia.

The number of clusters used in the model: This option is incorrect because the number of clusters is specified by the user, not represented by inertia.

Question 6/39

Given the following code snippet for performing hierarchical clustering in Python:

```
from scipy.cluster.hierarchy import linkage, dendrogram

import matplotlib.pyplot as plt


X = [[1, 2], [3, 4], [5, 6], [7, 8], [9, 10]]
```

```
Z = linkage(X, method='single', metric='euclidean')

dendrogram(Z)

plt.show()
```

Which part of the code specifies the method for determining the distance between clusters?

- [ ] metric='euclidean'
- [x] linkage(X, method='single', metric='euclidean')
- [ ] dendrogram(Z)
- [ ] X = [[1, 2], [3, 4], [5, 6], [7, 8], [9, 10]]

Explanation:

- linkage(X, method='single', metric='euclidean')

This option is correct because the linkage function call specifies both the method for determining the distance between clusters (method='single') and the metric for calculating distances between data points (metric='euclidean').

- dendrogram(Z)

This option is incorrect because dendrogram(Z) is the function call that generates the dendrogram plot.

- metric='euclidean'

This option is incorrect because metric='euclidean' specifies the distance metric used for calculating the distance between data points, not the method for clustering.

- X = [[1, 2], [3, 4], [5, 6], [7, 8], [9, 10]]

This option is incorrect because X = [[1, 2], [3, 4], [5, 6], [7, 8], [9, 10]] is simply the data being clustered, not the method for determining cluster distances.

Question 7/39

In hierarchical clustering, which linkage method considers the maximum distance between points in the clusters when merging two clusters?

- [ ] Ward linkage
- [x] Complete linkage
- [ ] Average linkage
- [ ] Single linkage

Explanation:

Complete linkage: This option is correct because complete linkage clustering, also known as farthest neighbour clustering, considers the maximum distance between points in the clusters when merging them.

Single linkage: This option is incorrect because single linkage considers the minimum distance between points in the clusters.

Average linkage: This option is incorrect because average linkage considers the average distance between points in the clusters.

Ward linkage: This option is incorrect because Ward linkage minimises the variance within each cluster when merging.

Question 8/39

Which of the following distance metrics is commonly used in hierarchical clustering?

- [ ] Hamming distance

- [ ] Jaccard index
- [x] Euclidean distance
- [ ] Cosine similarity

Explanation:

Euclidean distance: This option is correct because Euclidean distance is the most commonly used distance metric in hierarchical clustering for continuous data, as it measures the straight-line distance between points in the feature space.

Hamming distance: This option is incorrect because Hamming distance is typically used for categorical data, not for hierarchical clustering.

Cosine similarity: This option is incorrect because cosine similarity measures the cosine of the angle between two vectors and is not a distance metric.

Jaccard index: This option is incorrect because Jaccard index measures similarity between sample sets and is not typically used as a distance metric in hierarchical clustering.

Question 9/39

In agglomerative hierarchical clustering, what happens when two clusters are merged?

- [ ] The original clusters remain unchanged
- [ ] The number of clusters increases by one
- [x] The distance matrix is updated to reflect the new cluster
- [ ] The algorithm terminates

Explanation:

The distance matrix is updated to reflect the new cluster: This option is correct because when two clusters are merged in agglomerative clustering, the distance matrix must be updated to reflect the distances between the new cluster and the remaining clusters.

The number of clusters increases by one: This option is incorrect because merging two clusters reduces the total number of clusters by one.

The original clusters remain unchanged: This option is incorrect because merging means that the original clusters are combined into a single new cluster.

The algorithm terminates: This option is incorrect because the algorithm continues until only a single cluster remains.

Question 10/39

Consider the following code snippet:

```
X = [[1, 2], [2, 3], [3, 4], [5, 6], [7, 8]]


scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
model = AgglomerativeClustering(n_clusters=2, linkage='average')
model.fit(X_scaled)
```

Why do we use the fit_transform() method to scale the data?

- [ ] To assign cluster labels to each data point
- [ ] To increase the size of the dataset
- [ ] To reduce the number of features in the dataset

- [x] To ensure each feature contributes equally to the distance calculations

Explanation:

To ensure each feature contributes equally to the distance calculations: This option is correct because StandardScaler standardises the features by removing the mean and scaling to unit variance, ensuring that each feature contributes equally to the distance calculations in the clustering process.

To increase the size of the dataset: This option is incorrect because fit_transform() does not change the size of the dataset.

To reduce the number of features in the dataset: This option is incorrect because fit_transform() does not perform dimensionality reduction.

To assign cluster labels to each data point: This option is incorrect because assigning cluster labels is the job of the clustering algorithm, not the scaler.

Question 11/39

When calculating user similarity using cosine similarity, which matrix transformation is necessary before applying the similarity metric?

- [ ] Standardise the ratings.
- [ ] Fill NaN values with zeros.
- [ ] Normalise the ratings.
- [x] Transpose the utility matrix.

Explanation:

Transpose the utility matrix: This option is correct because transposing the utility matrix is necessary to align user vectors correctly for cosine similarity.

Normalise the ratings: This option is incorrect because normalisation is done to ensure consistent rating scales, but it is not specific to applying cosine similarity.

Fill NaN values with zeros: This option is incorrect because filling NaN values is a preprocessing step but not specific to the similarity calculation.

Standardise the ratings: This option is incorrect because standardisation is a type of normalisation, but transposition is specific to the similarity calculation step.

Question 12/39

Does the following code snippet correctly generate the top-N recommendations for a user using collaborative filtering?

```
def collab_generate_top_N_recommendations(user, N=10, k=20):
    if user not in user_sim_df.columns:
        return book_ratings.groupby('title').mean().sort_values(by='rating', ascending=False).index[:N].to_list()
    sim_users = user_sim_df.sort_values(by=user, ascending=False).index[1:k+1]
    favorite_user_items = []
    most_common_favorites = {}
    for i in sim_users:
        max_score = util_matrix_norm.loc[:, i].max()
        favorite_user_items.append(util_matrix_norm[util_matrix_norm.loc[:, i] == max_score].index.tolist())
    for item_collection in range(len(favorite_user_items)):
        for item in favorite_user_items[item_collection]:
            if item in most_common_favorites:
                most_common_favorites[item] += 1
            else:
                most_common_favorites[item] = 1
    sorted_list = sorted(most_common_favorites.items(), key=operator.itemgetter(1), reverse=True)[:N]
    top_N = [x[0] for x in sorted_list]
```

return top_N

- [x] Yes, it correctly generates top-N recommendations.
- [ ] No, it lacks the calculation of user similarity.
- [x] No, it does not consider the rating values.
- [ ] No, it does not handle the 'Cold-start problem' properly.

Explanation:

"Yes, it correctly generates top-N recommendations."
Explanation: This option is correct because the function calculates user similarity, considers rating values, and mitigates the 'Cold-start problem' by recommending popular items to new users.

"No, it lacks the calculation of user similarity."
This option is incorrect because user similarity is already calculated and utilised in sorting similar users.

"No, it does not consider the rating values."
This option is incorrect because the function does consider rating values by collecting max ratings from similar users.

"No, it does not handle the 'Cold-start problem' properly."
This option is incorrect because the function recommends popular items to mitigate for new users with no ratings.

Question 13/39

When generating rating predictions using collaborative filtering, what is the significance of the weighted average rating formula?

- [ ] It ensures all ratings are treated equally.

- [ ] It averages the ratings without considering similarity.
- [ ] It calculates the median rating.
- [x] It gives higher weight to ratings from more similar users.

Explanation:

"It gives higher weight to ratings from more similar users." This option is correct because in collaborative filtering, the weighted average formula assigns higher weights to ratings from users who are more similar to the target user, thus making the predictions more accurate.

"It ensures all ratings are treated equally." This option is incorrect because not all ratings are treated equally; similar users' ratings are more influential.

"It calculates the median rating." This option is incorrect because it does not calculate the median rating.

"It averages the ratings without considering similarity." This option is incorrect because similarity is crucial in the weighted average calculation.

Question 14/39

In a content-based filtering algorithm, what is the next step after computing the cosine similarity matrix?

- [ ] Predicting user ratings.
- [x] Generating a top-N list of recommendations.
- [ ] Splitting the dataset into train and test sets.

- [ ] Evaluating system performance using RMSE.

Explanation:

Generating a top-N list of recommendations.
This option is correct because after computing the cosine similarity matrix, the next step is to use the similarity scores to generate a top-N list of recommendations for each user based on their preferences.

Predicting user ratings.
This option is incorrect because predicting user ratings is typically done after generating recommendations.

Evaluating system performance using RMSE.
This option is incorrect because evaluating performance is usually done after recommendations are generated and assessed.

Splitting the dataset into train and test sets.
This option is incorrect because dataset splitting is an earlier step in the process.

Question 15/39

Which code snippet would you use to calculate the cosine similarity between all pairs of books in a dataset, given a TF-IDF matrix tfidf_matrix?

- [x] similarity_matrix = cosine_similarity(tfidf_matrix,tfidf_matrix)
- [ ] similarity_matrix = tfidf_matrix * tfidf_matrix.T
- [ ] similarity_matrix = np.dot(tfidf_matrix, tfidf_matrix.T)
- [ ] similarity_matrix = np.corrcoef(tfidf_matrix)

Explanation:

similarity_matrix = cosine_similarity(tfidf_matrix)

This option is correct because cosine_similarity(tfidf_matrix, tfidf_matrix) calculates the cosine similarity between all pairs of books based on their TF-IDF vectors, which is a common approach in content-based filtering.

similarity_matrix = np.dot(tfidf_matrix, tfidf_matrix.T)

This option is incorrect because the dot product does not compute cosine similarity.

similarity_matrix = np.corrcoef(tfidf_matrix)

This option is incorrect because np.corrcoef computes the Pearson correlation, not cosine similarity.

similarity_matrix = tfidf_matrix * tfidf_matrix.T

This option is incorrect because element-wise multiplication does not yield cosine similarity.

Question 16/39

True or false: In geospatial analysis, clustering algorithms can identify patterns in spatial data by grouping similar points based on their locations.

- [x] True
- [ ] False

Explanation:

This statement is true because clustering algorithms, such as K-means, DBSCAN, and hierarchical clustering, can analyse spatial data by grouping points that are close to each other, revealing patterns based on geographic proximity.

Question 17/39

Which Python library is specifically designed to extend Pandas with support for geographic data types and operations?

- [x] GeoPandas
- [ ] NumPy
- [ ] Pandas
- [ ] Matplotlib

Explanation:

- GeoPandas: GeoPandas is the correct answer because it extends Pandas to support geographic data types, enabling operations on spatial data like plotting maps and performing geometric computations.

- Pandas: This option is incorrect because Pandas is a data manipulation library that does not natively support geographic data types or spatial operations without additional extensions like GeoPandas.

- Matplotlib: This option is incorrect because Matplotlib is primarily a plotting library and does not extend Pandas with geographic data support.

- NumPy: This option is incorrect because NumPy focuses on numerical computations and does not provide specific support for geographic data types or operations.

Question 18/39

What is the main difference between a GeoDataFrame and a DataFrame?

- [ ] A GeoDataFrame is a subset of a DataFrame, designed specifically for geographical coordinates.

- [x] A DataFrame is used for handling tabular data, while a GeoDataFrame is specifically designed for geospatial data.
- [ ] A DataFrame allows for spatial operations, while a GeoDataFrame is limited to basic data manipulation.
- [ ] A GeoDataFrame can only handle numerical data, while a DataFrame can handle both numerical and categorical data.

Explanation:

- A DataFrame is used for handling tabular data, while a GeoDataFrame is specifically designed for geospatial data.
Correct. DataFrames handle general tabular data, while GeoDataFrames are for geospatial data with support for spatial operations.

- A GeoDataFrame can only handle numerical data, while a DataFrame can handle both numerical and categorical data.
Incorrect. Both can handle numerical and categorical data.

- A DataFrame allows for spatial operations, while a GeoDataFrame is limited to basic data manipulation.
Incorrect. It's the opposite; GeoDataFrames enable spatial operations.

- A GeoDataFrame is a subset of a DataFrame, designed specifically for geographical coordinates.
Misleading. A GeoDataFrame extends a DataFrame with geospatial capabilities, not merely a subset.

Question 19/39

True or false: The primary advantage of using DBSCAN for clustering in geospatial analysis is its ability to find clusters of varying shapes and sizes without specifying the number of clusters beforehand.

- [ ] False

- [x] True

Explanation:

This statement is true because DBSCAN (Density-Based Spatial Clustering of Applications with Noise) can identify clusters of varying shapes and sizes and does not require the number of clusters to be specified in advance.

Question 20/39

What column type is required in a GeoDataFrame to store the spatial features for each observation?

- [ ] Float
- [ ] String
- [x] GeoSeries
- [ ] Integer

Explanation:

- GeoSeries: This is the correct answer because a GeoSeries is a data structure within GeoPandas that is used to store spatial features (e.g., points, lines, polygons) for each observation in a GeoDataFrame.

- Integer: This option is incorrect because an Integer column cannot store spatial features, which are typically represented as geometric shapes rather than simple numeric values.

- String: This option is incorrect because a String column is used for textual data, not for spatial features, which require a specialised data structure like GeoSeries.

- Float: This option is incorrect because a Float column is used for floating-point numbers and

cannot store the complex geometric shapes needed for spatial data.

Question 21/39

Consider the following code snippet for clustering geospatial data using DBSCAN:

```
from sklearn.cluster import DBSCAN
import numpy as np

coordinates = np.array([
    [37.77, -122.42],
    [37.78, -122.41],
    [37.76, -122.43],
    [37.74, -122.44],
    [37.73, -122.45]
])

db = DBSCAN(eps=0.01, min_samples=2).fit(coordinates)
labels = db.labels_
print(labels)
```

What do the resulting labels indicate about the clustering of the coordinates?

- ✗ All points belong to the same cluster.
- ☐ All points are classified as noise.
- ☐ Each point is assigned to a unique cluster.
- ☐ Points are divided into clusters with at least one noise point.

Explanation:

- All points are classified as noise.
Correct: Given the small eps value of 0.01, it is highly unlikely that any points are within this distance from each other to form a cluster. The most likely outcome is that all points are too far apart to be considered part of a cluster, thus all being labelled as noise (-1).

- Points are divided into clusters with at least one noise point. This might be true in a different context but cannot be confirmed without knowing the labels the algorithm outputs.

- All points belong to the same cluster. This would only be correct if the eps value were sufficiently large to encompass all points in one cluster, which is unlikely given the small eps value of 0.01.

- Each point is assigned to a unique cluster. This suggests an eps too small to group any points together, which is a possible scenario given the eps of 0.01.

Question 22/39

True or false: The Expectation Maximisation (EM) algorithm in a Gaussian mixture model (GMM) is employed not only to derive the maximum likelihood estimates of the model parameters but also to ascertain the number of clusters that maximise the data likelihood.

- [ ] True
- [x] False

Explanation:

While the EM algorithm is utilised in GMMs to iteratively ascertain the maximum likelihood estimates of the parameters through the expectation (E) step and the maximisation (M) step, it does not determine the optimal number of clusters. The selection of the number of clusters, typically assessed through criteria such as Bayesian Information Criterion (BIC) or Akaike Information Criterion (AIC), is outside the direct scope of the EM algorithm.

Question 23/39

What role does the covariance matrix play in the Gaussian components of a GMM?

- [ ] It determines the mean of each component.
- [ ] It defines the number of components.
- [ ] It specifies the likelihood of each component.
- [x] It controls the width and orientation of each component.

Explanation:

- It controls the width and orientation of each component: Correct. The covariance matrix determines the shape, orientation, and spread of each Gaussian component in the mixture, effectively controlling how each component fits the data.

- It determines the mean of each component: Incorrect. The mean of each component is a separate parameter that specifies the centre of the Gaussian distribution, not its shape or orientation.

- It defines the number of components: Incorrect. The number of components is a separate parameter specified by the user or determined through model selection criteria, not by the covariance matrix.

It specifies the likelihood of each component: Incorrect. Each component's likelihood is influenced by its mean, covariance matrix, and weight, but the covariance matrix itself controls the shape and spread, not the likelihood directly.

Question 24/39

True or false: The sum of the weights of all Gaussian components in a GMM is always equal to one.

- [ ] False
- [x] True

Explanation:

In a GMM, the weights of all Gaussian components must sum to one, ensuring that the mixture model represents a valid probability distribution.

Question 25/39

Which method is commonly used to determine the optimal number of Gaussian components in a GMM?

- [ ] Cross-validation
- [x] Bayesian Information Criterion (BIC)
- [ ] Mean Squared Error (MSE)
- [ ] Silhouette score

Explanation:

- Bayesian Information Criterion (BIC): Correct. BIC is commonly used to select the optimal number of components by balancing model fit and complexity, penalising for the number of parameters to avoid overfitting.

- Cross-validation: Incorrect. While cross-validation is useful for model evaluation, it is not as commonly used for determining the number of components in GMMs compared to criteria like BIC or AIC.

- Silhouette score: Incorrect. The silhouette score is typically used for evaluating the quality of clusters in methods like K-means, not for determining the number of components in GMMs.

- Mean Squared Error (MSE): Incorrect. MSE is a

metric commonly used in regression analysis, not for selecting the number of components in GMMs.

Question 26/39

What is a primary advantage of using Gaussian mixture models (GMMs) for clustering?

- [ ] They are simpler to implement than other clustering algorithms.
- [ ] They require fewer computational resources compared to other methods.
- [x] They can model clusters with different shapes and sizes.
- [ ] They always produce spherical clusters.

Explanation:

- They can model clusters with different shapes and sizes: Correct. One of the main advantages of GMMs is their ability to model clusters that have varying shapes, sizes, and orientations, unlike methods like K-means which assume spherical clusters.

- They require fewer computational resources compared to other methods: Incorrect. GMMs can be computationally intensive, especially with high-dimensional data or a large number of components.

- They are simpler to implement than other clustering algorithms: Incorrect. GMMs are generally more complex to implement than simpler clustering algorithms like K-means due to the involvement of the EM algorithm and the need to estimate multiple parameters.

- They always produce spherical clusters: Incorrect. GMMs are capable of producing clusters of various shapes and are not restricted to spherical clusters, which is one of their primary advantages.

Question 27/39

Given the following code snippet that uses a Gaussian mixture model (GMM), what does the output of np.unique(labels, return_counts=True) represent?

```
from sklearn.mixture import GaussianMixture

import numpy as np

# Generating sample data

np.random.seed(0)

data = np.random.randn(300, 2)

# Fitting the Gaussian Mixture Model

gmm = GaussianMixture(n_components=3, random_state=0).fit(data)

labels = gmm.predict(data)

print(np.unique(labels, return_counts=True))
```

- [ ] The means and variances of the Gaussian components in the model.
- [x] It provides a summary of how many data points are assigned to each cluster label.
- [ ] The indices of the data points assigned to each cluster.
- [ ] The unique data points and their frequencies in the dataset.

Explanation:

- It provides a summary of how many data points are assigned to each cluster label. Correct: This option accurately describes what the output represents. np.unique with return_counts=True will return the unique labels and the count of each label in the dataset, showing how many data points belong to each of the clusters defined by the Gaussian mixture model.

- The unique data points and their frequencies in the dataset. This is incorrect. np.unique(labels,

return_counts=True) does not refer to the unique data points and their frequencies but rather to the unique labels (cluster assignments) and how many data points are assigned to each label.

- The indices of the data points assigned to each cluster. This is incorrect. The output does not provide the indices of the data points. It summarises the distribution of data points across the clusters by showing the unique labels and their counts.

- The means and variances of the Gaussian components in the model. This is incorrect. The means and variances of the Gaussian components are part of the model parameters and are not included in the output of np.unique(labels, return_counts=True). The correct method to obtain these parameters is by accessing gmm.means_ and gmm.covariances_.

Question 28/39

True or false: Advanced dimensionality reduction techniques can be both linear and non-linear.

- [x] True
- [ ] False

Explanation:

True: This is correct because advanced dimensionality reduction techniques encompass both linear methods, like Principal Component Analysis (PCA), and non-linear methods, like t-SNE (t-distributed Stochastic Neighbor Embedding) and Isomap. These techniques address different types of data structures and relationships.

Question 29/39

Which dimensionality reduction technique is known for using a probability distribution to measure the similarity between points in high-dimensional space?

- [ ] MDS
- [ ] LLE
- [ ] PCA
- [x] t-SNE

Explanation:

- t-SNE: This is correct. t-SNE (t-distributed Stochastic Neighbor Embedding) uses a probability distribution to measure the similarity between points in high-dimensional space and then attempts to preserve these similarities in a lower-dimensional space.

- PCA: This is incorrect. PCA (Principal Component Analysis) uses linear transformations to project data into a lower-dimensional space, focusing on preserving the variance, but does not use probability distributions for measuring similarity.

- MDS: This is incorrect. MDS (Multidimensional Scaling) focuses on preserving the pairwise distances or dissimilarities between points but does not specifically use a probability distribution for measuring similarities.

- LLE: This is incorrect. LLE (Locally Linear Embedding) is a non-linear dimensionality reduction technique that preserves local relationships using linear reconstructions, not probability distributions.

Question 30/39

True or false: In most cases, standardisation is a necessary preprocessing step before applying PCA to a dataset.

- [x] True
- [ ] False

Explanation:

True: This is generally correct because PCA (Principal Component Analysis) is sensitive to the scale of the data. Standardising the data (scaling to have zero mean and unit variance) ensures that each feature contributes equally to the analysis. However, there are specific scenarios where standardisation may be skipped if preserving the original scale of features is crucial for interpretation.

Question 31/39

True or False? Multidimensional Scaling (MDS) can only be applied to metric data.

- [ ] True
- [x] False

Explanation:

False: This is correct because MDS can be applied to both metric and non-metric data. Metric MDS uses actual distance values, while non-metric MDS uses rank orders of dissimilarities.

Question 32/39

True or false: t-SNE uses the Kullback–Leibler divergence to minimise the difference between the high-dimensional and low-dimensional data distributions.

- [x] True
- [ ] False

Explanation:

True: This is correct. t-SNE minimises the Kullback–Leibler divergence between the probability distributions of the high-dimensional data and the low-dimensional data to ensure that similar points in high-dimensional space stay close together in the low-dimensional representation.

Question 33/39

What will be the output of the code below if X is a dataset with 100 samples and 5 features, assuming all values are numeric?

```
from sklearn.decomposition import PCA

import matplotlib.pyplot as plt

# Assuming X is the input data matrix

pca = PCA(n_components=2)

X_pca = pca.fit_transform(X)

plt.scatter(X_pca[:, 0], X_pca[:, 1])

plt.show()
```

- [ ] A scatter plot with 5 points in a 100D space.
- [x] A scatter plot with 100 points in a 2D space.
- [ ] A scatter plot with 5 points in a 2D space.
- [ ] A scatter plot with 2 points in a 100D space.

Explanation:

- A scatter plot with 100 points in a 2D space. Correct. The PCA is used to reduce the dimensions of the dataset X from 5 features to 2 principal components. Therefore, the transformed data X_pca will have 100 samples with 2 features each, resulting in a scatter plot with 100 points in a 2D space.

- A scatter plot with 5 points in a 2D space. Incorrect. The dataset X has 100 samples, so the scatter plot will have 100 points, not 5.

- A scatter plot with 2 points in a 100D space.

Incorrect. The PCA reduces the dimensions to 2, and the scatter plot is in a 2D space, not 100D. Also, there are 100 samples, so it will not be 2 points.

- A scatter plot with 5 points in a 100D space. Incorrect. The scatter plot will be in a 2D space as PCA reduces the features to 2, and there are 100 samples, so it will have 100 points, not 5.

Question 34/39

True or False? Principal Component Analysis can be used for both supervised and unsupervised learning tasks.

- [x] True
- [ ] False

Explanation:

True: Correct. Principal Component Analysis (PCA) is primarily an unsupervised learning technique used for dimensionality reduction and exploratory data analysis. However, it can also be used as a preprocessing step in supervised learning tasks to reduce the number of features and improve model performance.

Question 35/39

What does the plot of eigenvalues in PCA indicate?

- [ ] It depicts the data points projected onto the principal components.
- [x] It displays the eigenvalues associated with each principal component.
- [ ] It shows the correlation between original features and principal components.

- [ ] It represents the cumulative variance explained by the principal components.

Explanation:

- It displays the eigenvalues associated with each principal component.
Correct: The plot (similar to a scree plot) shows the eigenvalues associated with each principal component, aiding in determining how many components to retain by showing the variance explained by each.

- It shows the correlation between original features and principal components.
Incorrect: The plot does not show correlations; it focuses on eigenvalues, which indicate the amount of variance each principal component captures.

- It represents the cumulative variance explained by the principal components.
Incorrect as the plot typically shows individual eigenvalues, not cumulative variance. Cumulative variance might be shown in a separate plot for clarity.

- It depicts the data points projected onto the principal components.
Incorrect: The plot does not visualise data points; it visualizes the eigenvalues of the principal components.

Question 36/39

In PCA, what is the main purpose of standardising the data before applying the algorithm?

- [ ] To reduce the number of features.
- [ ] To increase the computational complexity of the analysis.
- [ ] To enhance the convergence speed of the algorithm.
- [x] To ensure that features with larger scales

do not dominate the principal components.

Explanation:

To ensure that features with larger scales do not dominate the principal components: This is correct. Standardising the data ensures that each feature contributes equally to the analysis, preventing features with larger scales from dominating the principal components.

- To reduce the number of features: This is incorrect. Standardising does not reduce the number of features; it merely scales them to have a mean of zero and a standard deviation of one.

- To increase the computational complexity of the analysis: This is incorrect. Standardising the data does not aim to increase computational complexity; in fact, it can help improve numerical stability and efficiency.

- To enhance the convergence speed of the algorithm: This is incorrect because standardising data in PCA does not directly affect the convergence speed of the algorithm; however, it might be seen as a plausible but mistaken benefit since standardisation generally affects how algorithms process data.

Question 37/39

True or false: The first principal component in PCA captures the direction of the maximum variance in the data.

- [x] True
- [ ] False

Explanation:

True: Correct. The first principal component in PCA indeed captures the direction of the maximum variance in the data, providing the most significant linear combination of the original features.

Question 38/39

What is one practical application of PCA in data preprocessing?

- [ ] Automatically labelling unlabelled data.
- [ ] Generating synthetic data points for training.
- [x] Reducing overfitting by eliminating noise and less important features.
- [ ] Ensuring all data points belong to the same class.

Explanation:

- Reducing overfitting by eliminating noise and less important features: This is correct. PCA helps in reducing overfitting by transforming the data into a lower-dimensional space, capturing the most important features and eliminating noise.

- Generating synthetic data points for training: This is incorrect. PCA does not generate synthetic data points; it reduces the dimensionality of the existing data.

- Ensuring all data points belong to the same class: This is incorrect. PCA does not affect class labels or ensure uniformity of class membership; it is a method for dimensionality reduction.

- Automatically labelling unlabelled data: This is incorrect. PCA does not label data; it transforms data into principal components without altering class labels or providing automatic labelling.

Question 39/39

Consider the following Python code for applying PCA to a dataset using the scikit-learn library. Based on the output of the code, which statement is correct?

from sklearn.decomposition import PCA

```
import numpy as np

X = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9], [10, 11, 12]])

pca = PCA(n_components=2)

X_pca = pca.fit_transform(X)

print(X_pca)
```

- [ ] The number of principal components used is equal to the number of original features.
- [ ] PCA has been used to increase the number of features in the dataset.
- [ ] The output matrix will have more components than the original dataset.
- [x] The dataset has been transformed into a lower-dimensional space.

Explanation:

- The dataset has been transformed into a lower-dimensional space: This is correct. The code snippet indicates that PCA has been used to reduce the dimensionality from three features to two principal components.

- The number of principal components used is equal to the number of original features: This is incorrect. The PCA transformation reduces the number of dimensions from three to two, not maintaining the original number of features

- PCA has been used to increase the number of features in the dataset: This is incorrect. PCA is primarily used for reducing dimensionality, not increasing it

- The output matrix will have more components than the original dataset: This is incorrect. The output matrix will have fewer dimensions than the original dataset due to the reduction to two principal components.